

HTML QUICK - STYLER

A project report submitted to the NAAN MUDHALVAN COMMITTEE in
partial fulfilment of the requirement for the award of the **Degree of**

Bachelor of Science in Computer Science

Submitted by

S. ABINAYA MARY (23726ER001)

A. DEEPASANGEETHA (23726ER003)

P.GAYATHRI (23726ER007)

C.RADHAMANI (23726ER022)

Under the Guidance of

D.CHRISTY PRASANNA M.C.A., M.PHIL.

Assistant professor

Department of Computer Science



BON SECOURS ARTS AND SCIENCE COLLEGE FOR WOMEN

(Affiliated to Mother Teresa Women's University, Kodaikanal)

HTML quick- styler

PROJECT DESCRIPTION

The HTML Quick-Styler project is an AI-powered web page generation platform designed to simplify and accelerate the process of creating professional websites. It uses the IBM Granite 3.3 2B Instruct model to automatically generate structured HTML and CSS code based on user inputs such as page title, section requirements, and color preferences. The system analyzes these inputs and produces semantic HTML layouts with modern styling, responsive design, animations, and optimized structure, helping users build visually appealing web pages quickly.

The platform integrates technologies like Python, Gradio, HTML5, CSS3, and JavaScript to create an interactive and user-friendly interface for generating web pages. Through this interface, users can preview generated pages, select from multiple predefined color palettes, and customize sections such as hero banners, feature lists, portfolios, testimonials, and contact forms. The system also ensures responsive layouts and modern design standards so that the generated websites work effectively across different devices.

Overall, HTML Quick-Styler demonstrates how artificial intelligence can support web development by transforming simple user descriptions into fully structured and styled web pages. By combining AI capabilities with modern web technologies and cloud platforms like Google Colab, the project enables designers, developers, and beginners to create professional websites efficiently. The system highlights the potential of AI-assisted development to reduce development time while maintaining high-quality design and usability standards.

1.SYNOPSIS

HTML Quick-Styler is an intelligent web page generation system that helps users create modern HTML web pages easily. Many beginners and designers find it difficult to write HTML and CSS code manually. This project solves that problem by using artificial intelligence to automatically generate professional web pages from simple user descriptions.

The system uses the IBM Granite 3.3 2B Instruct model to understand the user's request. Users can provide inputs such as the page title, section details, and color style preferences. Based on this input, the system generates a structured HTML layout with modern CSS styling and responsive design.

Another important feature of the system is the automatic color palette generation. The system provides multiple design themes such as modern, vibrant, ocean, forest, and minimal. Users can preview the generated design and select their preferred style.

The platform is built using Python and the Gradio interface framework. The system runs on Google Colab and allows users to generate HTML pages quickly without writing complex code. This makes web development easier for both technical and non-technical users.

2.INTRODUCTION

Websites play a major role in communication, business, education, and online services. Creating a professional website usually requires knowledge of HTML, CSS, and JavaScript. For beginners, learning these technologies and designing web pages can be difficult and time-consuming.

The HTML Quick-Styler project was developed to simplify this process. It allows users to generate web pages automatically by simply describing what they want. The system uses artificial intelligence to understand the user's input and convert it into structured HTML and CSS code.

The system automatically detects sections such as hero banners, features, portfolios, testimonials, and contact forms. It then creates a complete web page with modern design elements and responsive layouts

By combining AI technology with modern web development tools, the HTML Quick-Styler platform provides a fast and user-friendly solution for creating websites.

3.SYSTEM SPECIFICATION

HARDWARE REQUIREMENTS

The HTML quick-styler does not require very high-end hardware because most of the processing is done on cloud platforms such as Google Colab.

Minimum Hardware Requirements:

- **Processor:** Intel i3 processor or equivalent
- **RAM:** Minimum 4 GB
- **Storage:** At least 1 GB of free disk space
- **Display:** Minimum screen resolution of 1280×720
- **Internet Connection:** Stable internet connection to access Google Colab and AI models.

SOFTWARE REQUIREMENTS

The following software components are required to develop and run the HTML quick-styler system.

- **Python 3.8 or higher** – Core programming language.
- **Google Colab** – Cloud platform used for development and execution.
- **Gradio** – Framework used to create the interactive interface
- **Transformers Library** – Used to load the IBM Granite AI model
- **PyTorch** – Deep learning framework required to running AI models
- **Web Browser** – Chrome, Firefox, Safari, or Edge to access the interface

3.1 SOFTWARE DESCRIPTION

Python

Python is the main programming language used in this project. It handles the application logic, user input processing, AI model interaction, and generation of HTML and CSS code.

IBM Granite 3.3 2B Instruct Model

The IBM Granite model is used to understand natural language inputs from users. It analyzes user descriptions and generates appropriate webpage structures.

Gradio

Gradio is used to create an interactive web interface for the system. It allows users to enter webpage details, preview generated pages, and select different design options.

HTML5 and CSS3

HTML5 and CSS3 are used to build the generated web pages. These technologies ensure responsive layouts, modern styling, and compatibility with different devices.

Google Colab

Google Colab is a cloud-based development environment used to run the project. It provides computing resources such as GPUs and allows the system to run without installing heavy software locally.

4.PROJECT OVERVIEW

4.1 SYSTEM ANALYSIS

System analysis is the process of understanding how the system should work and identifying the required features. The HTML Quick-Styler system focuses on three main aspects: webpage generation, design styling, and user interaction.

The user provides webpage details such as title, sections, and design preferences. The AI model processes this input and generates HTML structure and CSS styling. The generated page is displayed in the interface for preview and further customization.

Another important aspect of system analysis is ensuring that the system produces accurate and efficient results. The HTML Quick-Styler system analyzes the input data using the AI model and generates appropriate HTML and CSS code. It also ensures that the generated webpage follows modern design standards such as responsive layouts, proper section organization, and attractive color themes. Through this process, the system provides a reliable and user-friendly platform for creating web pages quickly and efficiently.

4.2 OVERVIEW OF PROJECT IN PHASES

Phase1: Environment Setup & Model Configuration

Install Python libraries, configure Google Colab environment, and load the IBM Granite AI model.

Phase 2: Core Engine Development

Develop the HTML generation engine, create color palette modules, and implement CSS generation logic.

Phase 3: Interface Development

Create a user-friendly Gradio interface that allows users to generate and preview web pages.

Phase 4: Integration & Testing

Integrate all modules and test the system to ensure accurate webpage generation.

Phase 5: Deployment & Documentation

Deploy the project on Google Colab and prepare documentation for users.

5. MODULES AND MODULES DESCRIPTIONS

This module generates a structured HTML webpage based on the user's input description. It automatically organizes sections and applies styling to create a complete webpage.

1. User Interface Module

Description:

- Provides an interactive interface for users to enter webpage details.
- Built using the Gradio framework.
- Displays generated HTML and preview results.

Key Features:

- Input form for page title and description.
- Webpage preview section.
- Template and palette selection.

2. AI Content Generation Module

Description:

- Uses IBM Granite AI model to analyze and interpret user input.
- Identifies webpage sections automatically.

Key Features:

- Natural language understanding
- Section detection
- Content suggestions

3. HTML Page Builder Module

Description:

- Generates structured HTML code for different webpage sections

Key Features:

- Hero section generation
- Portfolio layout creation
- Contact form generation

4. CSS Styling Module

Description:

- Generates modern CSS styles automatically

Key Features:

- Responsive design
- Animation effects
- Grid and Flexbox layouts

5. Color Palette

Description:

- Provides predefined color themes for webpage design

Key Features:

- Modern palette

6. SAMPLE CODING

```
# -*- coding: utf-8 -*-  
"""Untitled0.ipynb
```

Automatically generated by Colab.

Original file is located at

<https://colab.research.google.com/drive/1XEAYMuW5f1R1bRdcSNI0pkjHhzbo9SLR>

```
!pip install -U transformers  
"""
```

```
!pip install -U transformers
```

```
# use a pipeline as a high-level helper  
from transformers import pipeline
```

```
pipe = pipeline("text-generation", model="ibm-granite/granite-3.3-2b-instruct")  
messages = [  
    {"role": "user", "content": "who are you?"},  
]  
pipe(messages)
```

```
!pip install -q gradio
```

```
import gradio as gr  
import json  
import re  
from datetime import datetime  
from typing import Tuple, Dict, List  
import random
```

```
#  
=====
```

```
# HTML QUICK-STYLER - ADVANCED VERSION (FIXED)
```

```
#  
=====
```

```
class ColorPaletteGenerator:
```

""Generate visually cohesive color palettes""

@staticmethod

def generate_palette(style: str = "modern") -> Dict[str, str]:

""Generate a cohesive color palette based on style""

```
palettes = {
    'modern': {
        'primary': '#667eea',
        'secondary': '#764ba2',
        'accent': '#f093fb',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'vibrant': {
        'primary': '#ff6b6b',
        'secondary': '#ff8e72',
        'accent': '#f5576c',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'ocean': {
        'primary': '#0099cc',
        'secondary': '#006699',
        'accent': '#00ccff',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
    'forest': {
        'primary': '#27ae60',
        'secondary': '#1e8449',
        'accent': '#52be80',
        'background': '#0f172a',
        'surface': '#1e293b',
        'text_primary': '#f1f5f9',
        'text_secondary': '#cbd5e1',
        'border': '#334155'
    },
}
```

```

'sunset': {
    'primary': '#ff8c42',
    'secondary': '#ff6b35',
    'accent': '#ffa751',
    'background': '#0f172a',
    'surface': '#1e293b',
    'text_primary': '#f1f5f9',
    'text_secondary': '#cbd5e1',
    'border': '#334155'
},
'luxury': {
    'primary': '#1a1a1a',
    'secondary': '#333333',
    'accent': '#FFD700',
    'background': '#0f172a',
    'surface': '#1e293b',
    'text_primary': '#f1f5f9',
    'text_secondary': '#cbd5e1',
    'border': '#334155'
},
'creative': {
    'primary': '#9b59b6',
    'secondary': '#8e44ad',
    'accent': '#e74c3c',
    'background': '#0f172a',
    'surface': '#1e293b',
    'text_primary': '#f1f5f9',
    'text_secondary': '#cbd5e1',
    'border': '#334155'
},
'minimal': {
    'primary': '#2c3e50',
    'secondary': '#34495e',
    'accent': '#3498db',
    'background': '#0f172a',
    'surface': '#1e293b',
    'text_primary': '#f1f5f9',
    'text_secondary': '#cbd5e1',
    'border': '#334155'
}
}

```

```

return palettes.get(style.lower(), palettes['modern'])

```

```

class HTMLPageBuilder:

```

```

    """Build styled HTML pages from descriptions"""

```

```

@staticmethod
def extract_sections(description: str) -> List[str]:
    """Extract sections from description"""
    sections = []

    section_keywords = {
        'hero': ['hero', 'banner', 'landing'],
        'features': ['features', 'services', 'benefits'],
        'about': ['about', 'team', 'company'],
        'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
        'testimonials': ['testimonial', 'reviews', 'feedback'],
        'pricing': ['pricing', 'plans', 'cost'],
        'contact': ['contact', 'form', 'reach'],
        'footer': ['footer', 'bottom']
    }

    desc_lower = description.lower()

    for section_type, keywords in section_keywords.items():
        for keyword in keywords:
            if keyword in desc_lower:
                sections.append(section_type)
                break

    if not sections:
        sections = ['hero', 'features', 'contact', 'footer']

    return sections

```

```

@staticmethod
def generate_meta_tags(title: str, description: str) -> str:
    """Generate SEO meta tags"""
    meta_tags = f"    <meta charset='UTF-8'>
<meta name='viewport' content='width=device-width, initial-scale=1.0'>
<title>{title}</title>
<meta name='description' content='{description[:160]}'>
<meta name='theme-color' content='#0f172a'>
<meta property='og:title' content='{title}'>
<meta property='og:description' content='{description[:160]}'>
<meta property='og:type' content='website'>
<meta name='twitter:card' content='summary_large_image'>
<meta name='author' content='HTML Quick-Styler'>"
    return meta_tags

```

```

@staticmethod
def generate_section_html(section_type: str, palette: Dict) -> str:
    """Generate HTML for specific section"""

```

```

sections = {
  'hero': f"    <section class='hero'>
    <div class='hero-content'>
      <h1>Welcome to Your Site</h1>
      <p>Create amazing web experiences</p>
      <button class='btn btn-primary'>Get Started</button>
    </div>
  </section>",

  'features': f"    <section class='features'>
    <h2>Features</h2>
    <div class='features-grid'>
      <div class='feature-card'>
        <h3>Fast</h3>
        <p>Lightning-quick performance</p>
      </div>
      <div class='feature-card'>
        <h3>Secure</h3>
        <p>Enterprise-grade security</p>
      </div>
      <div class='feature-card'>
        <h3>Scalable</h3>
        <p>Grow without limits</p>
      </div>
    </div>
  </section>",

  'about': f"    <section class='about'>
    <div class='about-content'>
      <h2>About Us</h2>
      <p>We're passionate about creating beautiful, functional websites that
help your business grow.</p>
    </div>
  </section>",

  'portfolio': f"    <section class='portfolio'>
    <h2>Our Work</h2>
    <div class='portfolio-grid'>
      <div class='portfolio-item'></div>
      <div class='portfolio-item'></div>
      <div class='portfolio-item'></div>
    </div>
  </section>",

  'testimonials': f"    <section class='testimonials'>
    <h2>What People Say</h2>
    <div class='testimonials-grid'>

```

```
<div class="testimonial">
  <p>"Amazing service!"</p>
  <span>- Client Name</span>
</div>
<div class="testimonial">
  <p>"Highly recommended!"</p>
  <span>- Happy Customer</span>
</div>
</div>
</section>"
```

```
'pricing': f"    <section class="pricing">
<h2>Pricing Plans</h2>
<div class="pricing-grid">
  <div class="pricing-card">
    <h3>Starter</h3>
    <p class="price">$29</p>
    <ul><li>Feature 1</li><li>Feature 2</li></ul>
  </div>
  <div class="pricing-card featured">
    <h3>Pro</h3>
    <p class="price">$79</p>
    <ul><li>All Starter</li><li>Feature 3</li></ul>
  </div>
</div>
</section>"
```

```
'contact': f"    <section class="contact">
<h2>Get in Touch</h2>
<form class="contact-form">
  <input type="text" placeholder="Your Name" required>
  <input type="email" placeholder="Your Email" required>
  <textarea placeholder="Your Message" rows="5"
required></textarea>
  <button type="submit" class="btn btn-primary">Send
Message</button>
</form>
</section>"
```

```
'footer': f"    <footer class="footer">
<div class="footer-content">
  <p>&copy; 2024 Your Company. All rights reserved.</p>
  <div class="footer-links">
    <a href="#privacy">Privacy</a>
    <a href="#terms">Terms</a>
    <a href="#contact">Contact</a>
  </div>
```



```
    </div>
</footer>"""
}
```

```
return sections.get(section_type, "")
```

```
@staticmethod
```

```
def generate_css(palette: Dict) -> str:
```

```
    """Generate comprehensive CSS"""
```

```
    css = f""" <style>
```

```
    * {{
```

```
        margin: 0;
```

```
        padding: 0;
```

```
        box-sizing: border-box;
```

```
    }}
```

```
:root {{
```

```
    --primary: {palette['primary']};
```

```
    --secondary: {palette['secondary']};
```

```
    --accent: {palette['accent']};
```

```
    --background: {palette['background']};
```

```
    --surface: {palette['surface']};
```

```
    --text-primary: {palette['text_primary']};
```

```
    --text-secondary: {palette['text_secondary']};
```

```
    --border: {palette['border']};
```

```
}}
```

```
body {{
```

```
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
    background: linear-gradient(135deg, var(--background) 0%, var(--surface) 100%);
```

```
    color: var(--text-primary);
```

```
    line-height: 1.6;
```

```
    min-height: 100vh;
```

```
}}
```

```
header {{
```

```
    background: rgba(15, 23, 42, 0.95);
```

```
    backdrop-filter: blur(10px);
```

```
    padding: 1.5rem 2rem;
```

```
    position: sticky;
```

```
    top: 0;
```

```
    z-index: 100;
```

```
    border-bottom: 1px solid var(--border);
```

```
}}
```

```
header nav {{
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    max-width: 1200px;
    margin: 0 auto;
  }}

header h1 {{
  font-size: 1.8rem;
  background: linear-gradient(135deg, var(--primary), var(--accent));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}}

header nav a {{
  color: var(--text-secondary);
  text-decoration: none;
  margin: 0 1.5rem;
  transition: color 0.3s ease;
}}

header nav a:hover {{
  color: var(--primary);
}}

main {{
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 2rem;
}}

section {{
  padding: 4rem 0;
  animation: fadeIn 0.6s ease-in;
}}

@keyframes fadeIn {{
  from {{
    opacity: 0;
    transform: translateY(20px);
  }}
  to {{
    opacity: 1;
    transform: translateY(0);
  }}
}}
```

```
h1, h2, h3 {{
  color: var(--text-primary);
  margin-bottom: 1rem;
}}

h1 {{
  font-size: 3rem;
  font-weight: 700;
}}

h2 {{
  font-size: 2rem;
  margin-bottom: 2rem;
  text-align: center;
}}

h3 {{
  font-size: 1.3rem;
}}

.hero {{
  background: linear-gradient(135deg, var(--primary) 0%, var(--
secondary) 100%);
  color: white;
  padding: 6rem 0;
  text-align: center;
  min-height: 500px;
  display: flex;
  align-items: center;
  justify-content: center;
}}

.hero h1 {{
  color: white;
  font-size: 4rem;
  margin-bottom: 1rem;
  animation: slideDown 0.8s ease;
}}

.hero p {{
  font-size: 1.5rem;
  margin-bottom: 2rem;
  opacity: 0.95;
  animation: slideUp 0.8s ease;
}}
```

```
@keyframes slideDown {{
  from {{
    opacity: 0;
    transform: translateY(-30px);
  }}
  to {{
    opacity: 1;
    transform: translateY(0);
  }}
}}
```

```
@keyframes slideUp {{
  from {{
    opacity: 0;
    transform: translateY(30px);
  }}
  to {{
    opacity: 1;
    transform: translateY(0);
  }}
}}
```

```
.btn {{
  padding: 0.8rem 2rem;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  cursor: pointer;
  transition: all 0.3s ease;
  font-weight: 600;
}}
```

```
.btn-primary {{
  background: var(--primary);
  color: white;
  box-shadow: 0 4px 15px rgba(0,0,0,0.3);
}}
```

```
.btn-primary:hover {{
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(0,0,0,0.4);
  background: var(--accent);
}}
```

```
.btn-secondary {{
  background: var(--secondary);
  color: white;
```

```
}}
```

```
.btn-secondary:hover {{  
  background: var(--primary);  
}}
```

```
.features-grid {{  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: 2rem;  
}}
```

```
.feature-card {{  
  background: rgba(30, 41, 59, 0.8);  
  padding: 2rem;  
  border-radius: 12px;  
  border: 1px solid var(--border);  
  transition: all 0.3s ease;  
  backdrop-filter: blur(10px);  
}}
```

```
.feature-card:hover {{  
  transform: translateY(-8px);  
  border-color: var(--primary);  
  box-shadow: 0 20px 40px rgba(0,0,0,0.3);  
}}
```

```
.feature-card h3 {{  
  color: var(--primary);  
  margin-bottom: 0.5rem;  
}}
```

```
.feature-card p {{  
  color: var(--text-secondary);  
}}
```

```
.portfolio-grid {{  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));  
  gap: 2rem;  
}}
```

```
.portfolio-item {{  
  aspect-ratio: 1;  
  background: linear-gradient(135deg, var(--primary), var(--secondary));  
  border-radius: 12px;  
  cursor: pointer;
```

```
    transition: all 0.3s ease;
    border: 1px solid var(--border);
  }
```

```
.portfolio-item:hover {
  transform: scale(1.05);
  box-shadow: 0 20px 40px rgba(0,0,0,0.3);
}
```

```
.about {
  text-align: center;
}
```

```
.about-content {
  max-width: 800px;
  margin: 0 auto;
}
```

```
.about p {
  font-size: 1.1rem;
  color: var(--text-secondary);
}
```

```
.testimonials-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 2rem;
}
```

```
.testimonial {
  background: rgba(30, 41, 59, 0.8);
  padding: 2rem;
  border-left: 4px solid var(--primary);
  border-radius: 8px;
}
```

```
.testimonial p {
  font-style: italic;
  margin-bottom: 1rem;
  color: var(--text-secondary);
}
```

```
.testimonial span {
  font-weight: 600;
  color: var(--primary);
}
```

```
.pricing-grid {{
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 2rem;
}}
```

```
.pricing-card {{
  background: rgba(30, 41, 59, 0.8);
  padding: 2rem;
  border-radius: 12px;
  border: 1px solid var(--border);
  text-align: center;
  transition: all 0.3s ease;
}}
```

```
.pricing-card.featured {{
  border-color: var(--primary);
  box-shadow: 0 20px 40px rgba(0,0,0,0.3);
  transform: scale(1.05);
}}
```

```
.pricing-card .price {{
  font-size: 2.5rem;
  color: var(--primary);
  margin: 1rem 0;
  font-weight: 700;
}}
```

```
.pricing-card ul {{
  list-style: none;
  margin: 1.5rem 0;
  text-align: left;
}}
```

```
.pricing-card li {{
  padding: 0.5rem 0;
  color: var(--text-secondary);
}}
```

```
.contact {{
  max-width: 600px;
  margin: 0 auto;
}}
```

```
.contact-form {{
  display: flex;
  flex-direction: column;
```

```
    gap: 1.5rem;
  }}
```

```
.contact-form input,
.contact-form textarea {{
  padding: 1rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  background: rgba(30, 41, 59, 0.8);
  color: var(--text-primary);
  font-family: inherit;
  font-size: 1rem;
}}
```

```
.contact-form input:focus,
.contact-form textarea:focus {{
  outline: none;
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(0,0,0,0.3);
}}
```

```
.footer {{
  background: rgba(15, 23, 42, 0.95);
  border-top: 1px solid var(--border);
  margin-top: 4rem;
  padding: 3rem 0;
}}
```

```
.footer-content {{
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
  gap: 2rem;
}}
```

```
.footer-links {{
  display: flex;
  gap: 2rem;
}}
```

```
.footer-links a {{
  color: var(--text-secondary);
  text-decoration: none;
```



```

        transition: color 0.3s ease;
    }}

.footer-links a:hover {{
    color: var(--primary);
}}

@media (max-width: 768px) {{
    header nav {{
        flex-direction: column;
        gap: 1rem;
    }}

    header h1 {{
        font-size: 1.4rem;
    }}

    .hero h1 {{
        font-size: 2rem;
    }}

    .hero p {{
        font-size: 1.1rem;
    }}

    h2 {{
        font-size: 1.5rem;
    }}

    main {{
        padding: 0 1rem;
    }}

    section {{
        padding: 2rem 0;
    }}

    .footer-content {{
        flex-direction: column;
        text-align: center;
    }}
}}
</style>"""
return css

```

```

class HTMLGenerationEngine:
    """Main engine for HTML page generation"""

```

```

@staticmethod
def generate_page(title: str, description: str, style: str = "modern") ->
Tuple[str, str]:
    """Generate complete HTML page"""

    # Generate color palette
    palette = ColorPaletteGenerator.generate_palette(style)

    # Extract sections
    sections = HTMLPageBuilder.extract_sections(description)

    # Generate meta tags
    meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

    # Generate sections
    sections_html = '\n\n'.join([
        HTMLPageBuilder.generate_section_html(section, palette)
        for section in sections
        if HTMLPageBuilder.generate_section_html(section, palette)
    ])

    # Generate CSS
    css = HTMLPageBuilder.generate_css(palette)

    # Build complete HTML
    html = f"<!DOCTYPE html>
<html lang="en">
<head>
{meta_tags}
{css}
</head>
<body>
<header>
<nav>
<h1>{title}</h1>
<div>
<a href="#features">Features</a>
<a href="#about">About</a>
<a href="#contact">Contact</a>
</div>
</nav>
</header>

<main>
{sections_html}
</main>

```

```
</body>
</html>"""
```

```
    return html, palette
```

```
generation_history = []
```

```
def generate_html_page(title: str, description: str, style: str = "modern") ->
    Tuple[str, str, str]:
```

```
    """Generate HTML page with styling"""
```

```
    if not title.strip() or not description.strip():
```

```
        return "", "✗ Please provide both title and description!", ""
```

```
    try:
```

```
        html_output, palette = HTMLGenerationEngine.generate_page(title,
description, style)
```

```
        color_info = f"""
```

```
**Color Palette:**
```

```
- Primary: {palette['primary']}
```

```
- Secondary: {palette['secondary']}
```

```
- Accent: {palette['accent']}
```

```
- Background: {palette['background']}
```

```
"""
```

```
        generation_history.append({
            'title': title,
            'description': description,
            'html': html_output,
            'palette': palette,
            'timestamp': datetime.now().isoformat()
        })
```

```
        return html_output, "✓ HTML page generated successfully!", color_info
    except Exception as e:
```

```
        return "", f"✗ Error: {str(e)}", ""
```

```
def get_color_suggestions(style: str) -> str:
```

```
    """Get color suggestions based on style"""
```

```
    palette = ColorPaletteGenerator.generate_palette(style)
```

```
    palette_text = f"""
```

```
**Palette for "{style}" style:**
```

```
- Primary: {palette['primary']}
```

```
- Secondary: {palette['secondary']}
```

```
- Accent: {palette['accent']}
```

```
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
```

```
    return palette_text
```

```
print("🚀 Loading HTML Quick-Styler Advanced...")
```

```
with gr.Blocks(title="HTML Quick-Styler - Advanced Edition") as demo:
```

```
    gr.Markdown("""
    # 🚀 HTML Quick-Styler Advanced
    ## Professional HTML Page Generator with Auto-Styling

    **Generate production-ready HTML pages in seconds**
    """)
```

```
    with gr.Tabs():
```

```
        # ===== TAB 1: Page Generator =====
```

```
        with gr.TabItem("🔧 Generate Page"):
```

```
            with gr.Row():
```

```
                with gr.Column(scale=1):
```

```
                    gr.Markdown("### 📄 Page Details")
```

```
                    page_title = gr.Textbox(
                        label="Page Title",
                        value="Modern Portfolio Website",
                        lines=1
                    )
```

```
                    page_description = gr.Textbox(
                        label="Page Description & Sections",
                        value="Hero section with welcome message, features list with 3
items, about section, portfolio gallery, testimonials, contact form, and footer",
                        lines=4
                    )
```

```
                    color_style = gr.Dropdown(
                        ["modern", "vibrant", "ocean", "forest", "sunset", "luxury",
"creative", "minimal"],
                        value="modern",
                        label="🎨 Color Style"
                    )
```

```
                    generate_btn = gr.Button("⚡ Generate HTML", size="lg",
variant="primary")
```

```

with gr.Column(scale=1):
    gr.Markdown("### 🖨️ Output")

    html_output = gr.Code(
        language="html",
        label="Generated HTML/CSS",
        lines=20
    )

    status_msg = gr.Markdown()
    color_info = gr.Markdown()

generate_btn.click(
    generate_html_page,
    inputs=[page_title, page_description, color_style],
    outputs=[html_output, status_msg, color_info]
)

# ===== TAB 2: Preview =====
with gr.TabItem("👁️ Live Preview"): gr.Markdown("###  
Q Preview Generated Page")

    preview_button = gr.Button("🔄 Load Preview", size="lg",
variant="primary")
    preview_output = gr.HTML(value="<p style='text-align:center;  
color:#fff; padding:40px;'>No preview loaded. Generate a page first.</p>")

    def show_preview(html_text):
        if not html_text.strip():
            return "<p style='text-align:center; color:#fff; padding:40px;'>No  
HTML provided</p>"
        return html_text

    preview_button.click(
        show_preview,
        inputs=[html_output],
        outputs=[preview_output]
    )

# ===== TAB 3: Color Palettes =====
with gr.TabItem("🎨 Color Palettes"):
    gr.Markdown("### 🌈 Explore Color Schemes")

    with gr.Row():
        style_selector = gr.Radio(

```

```

        ["modern", "vibrant", "ocean", "forest", "sunset", "luxury",
"creative", "minimal"],
        value="modern",
        label="Choose Style"
    )

```

```

palette_btn = gr.Button("🎨 Get Palette", size="lg",
variant="primary")

```

```

palette_display = gr.Markdown()

```

```

palette_btn.click(
    get_color_suggestions,
    inputs=[style_selector],
    outputs=[palette_display]
)

```

```

# ===== TAB 4: Templates =====

```

```

with gr.TabItem("📁 Templates"):

```

```

    gr.Markdown("### 🚀 Quick Templates")

```

```

    with gr.Row():

```

```

        with gr.Column():

```

```

            gr.Markdown("""

```

```

            ##### Portfolio Website

```

```

            Hero, Portfolio Gallery, About, Contact, Footer

```

```

            """)

```

```

            portfolio_btn = gr.Button("Use Template", size="sm")

```

```

        def load_portfolio():

```

```

            return ("Portfolio Website",

```

```

                    "Hero section with welcome banner, portfolio gallery with 6
items, about section, testimonials, contact form, footer",
                    "modern")

```

```

            portfolio_btn.click(load_portfolio, outputs=[page_title,
page_description, color_style])

```

```

    with gr.Column():

```

```

        gr.Markdown("""

```

```

        ##### SaaS Landing Page

```

```

        Hero, Features, Pricing, Testimonials, CTA, Footer

```

```

        """)

```

```

        saas_btn = gr.Button("Use Template", size="sm")

```

```

        def load_saas():

```

```

        return ("SaaS Product",
                "Hero banner, features showcase with 3 items, pricing plans
comparison, testimonials, call-to-action button, footer",
                "vibrant")

```

```

saas_btn.click(load_saas, outputs=[page_title, page_description,
color_style])

```

```

with gr.Column():
    gr.Markdown("""
#### Company Website
Header, About, Services, Team, Contact, Footer
""")
    company_btn = gr.Button("Use Template", size="sm")

```

```

def load_company():
    return ("Company",
            "Header with navigation, hero section, about company,
services list, team members, contact form, footer",
            "ocean")

```

```

company_btn.click(load_company, outputs=[page_title,
page_description, color_style])

```

```

# ===== TAB 5: Guide =====

```

```

with gr.TabItem("📖 Guide"):
    gr.Markdown("""
# 📖 HTML Quick-Styler Guide

```

```

## How It Works

```

```

### 1. Enter Details

```

- **Title:** Page name
- **Description:** Sections you want
- **Color Style:** Pick a theme

```

### 2. Automatic Detection

```

The tool recognizes these keywords:

- `hero` / `banner` → Hero Section
- `features` / `services` → Features Grid
- `about` / `team` → About Section
- `portfolio` / `gallery` → Portfolio
- `testimonial` / `reviews` → Testimonials
- `pricing` / `plans` → Pricing Plans
- `contact` / `form` → Contact Form
- `footer` → Footer

3. Generate & Customize

- Copy generated HTML
- Edit text and colors
- Save as .html file
- Open in browser

Features

- ✓ Responsive design
- ✓ Modern animations
- ✓ SEO meta tags
- ✓ Professional styling
- ✓ Dark theme
- ✓ Color palettes
- ✓ Copy-paste ready

Color Styles

- **Modern:** Professional blues
 - **Vibrant:** Bold reds
 - **Ocean:** Calm blues
 - **Forest:** Natural greens
 - **Sunset:** Warm oranges
 - **Luxury:** Elegant blacks
 - **Creative:** Purple artistic
 - **Minimal:** Clean grays
- """)

```
print("\n✓ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True)
```

```
# Load model directly
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```
tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
```

```
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
```

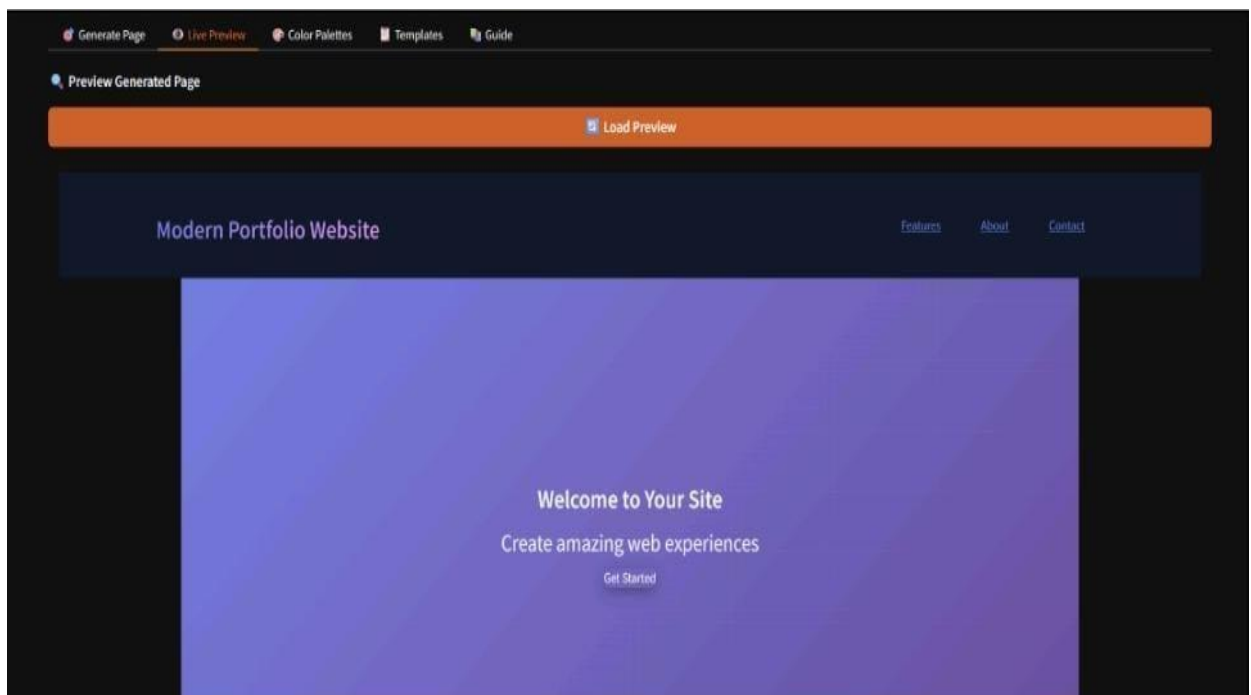
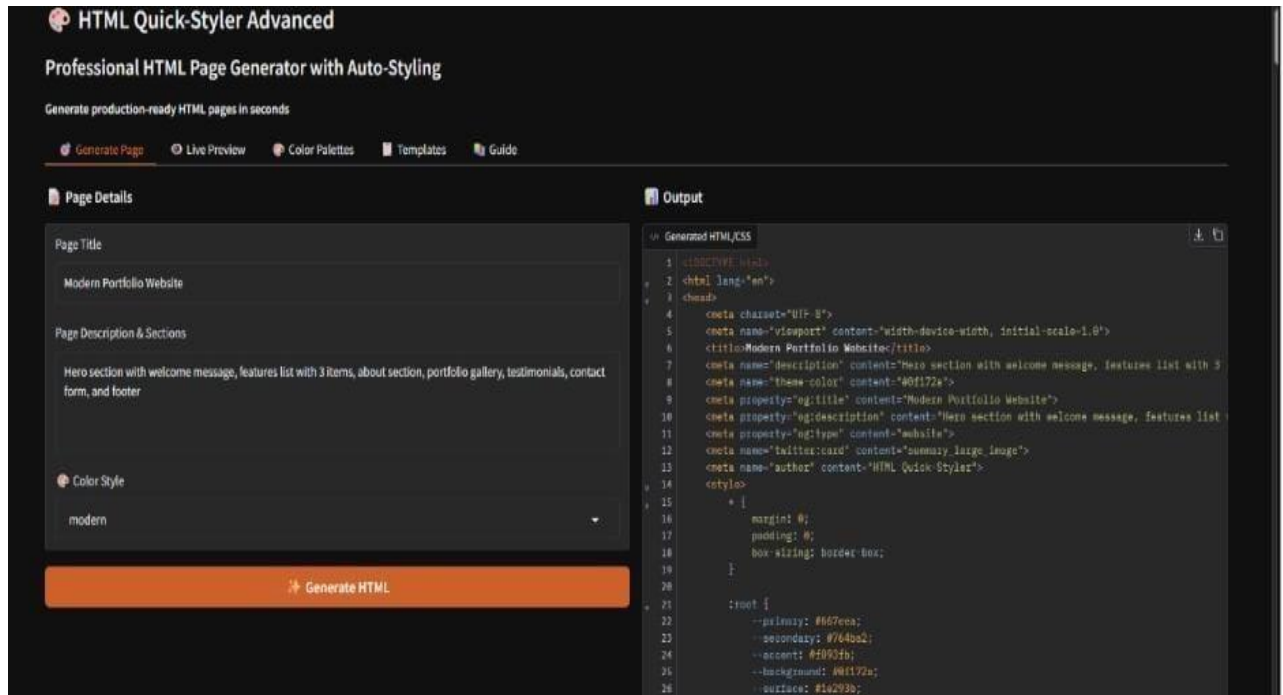
```
messages = [
    {"role": "user", "content": "Who are you?"},
]
```

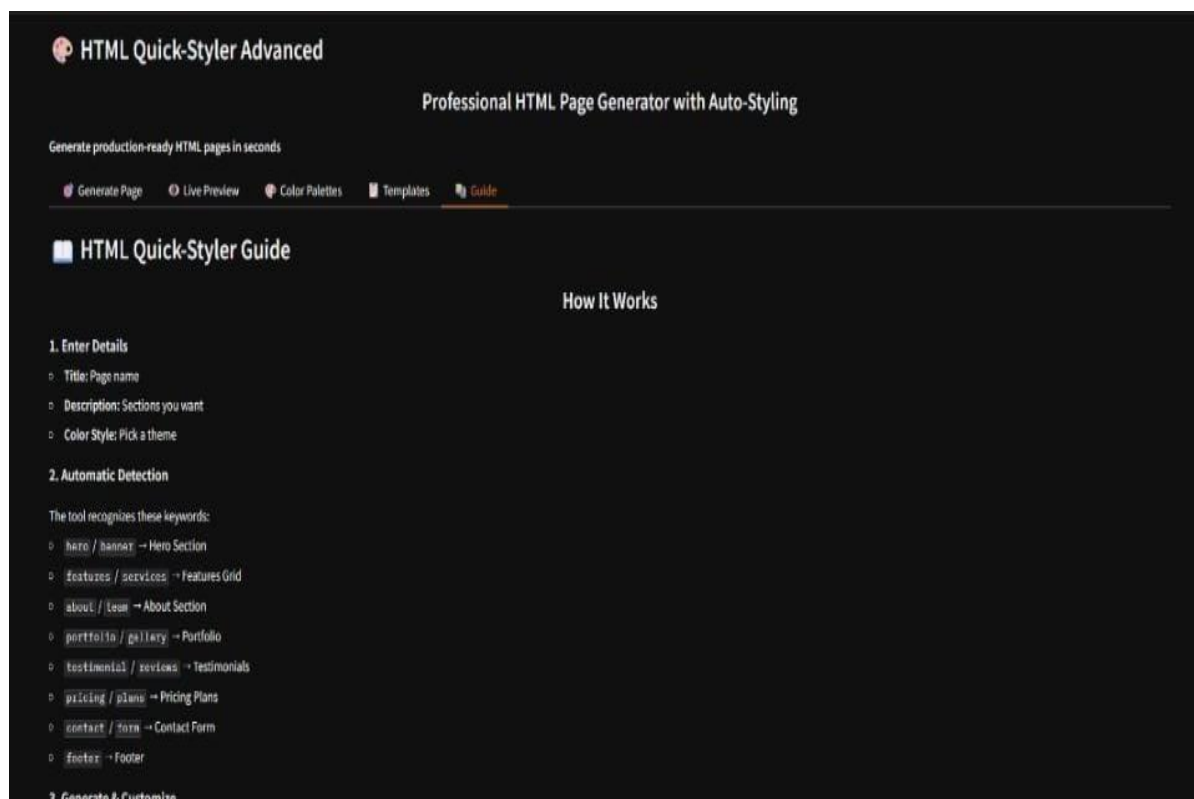
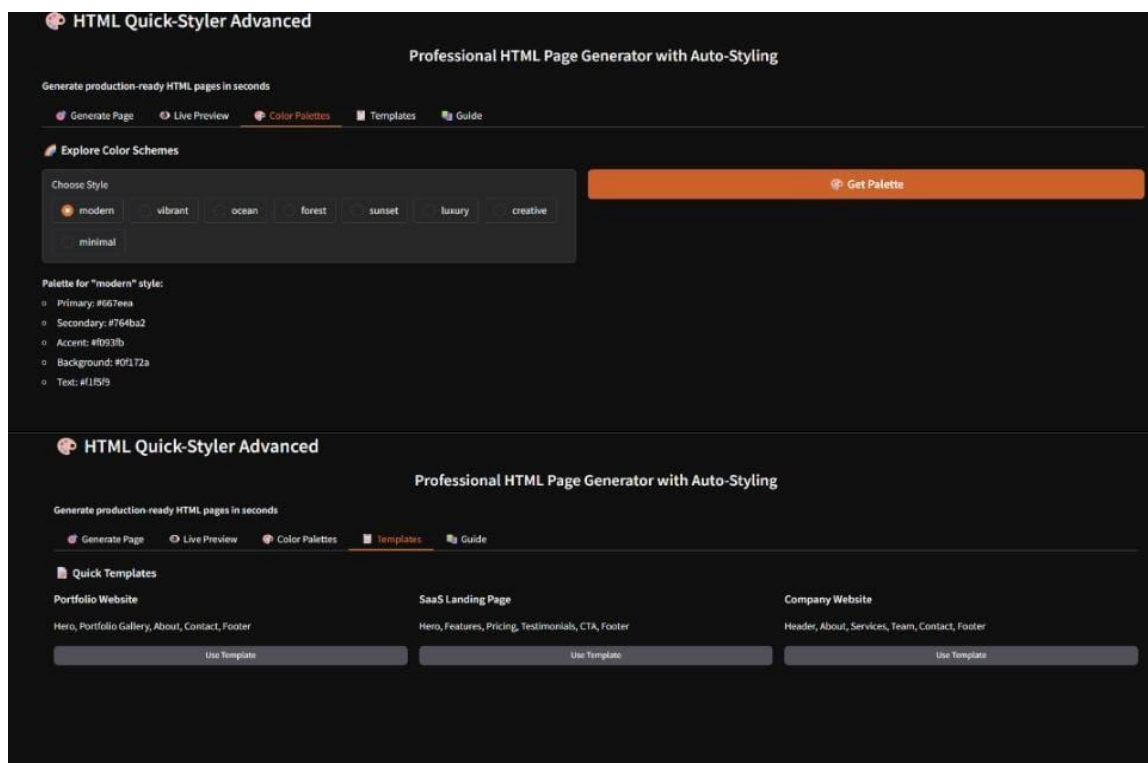
```
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
```



```
        tokenize=True,  
        return_dict=True,  
        return_tensors="pt",  
    ).to(model.device)  
  
outputs = model.generate(**inputs, max_new_tokens=40)  
  
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:])
```

7. OUTPUT SCREENSHOT





8. CONCLUSION

The HTML Quick-Styler project demonstrates how artificial intelligence can simplify the web development process. The system allows users to generate professional web pages quickly by providing simple descriptions.

By combining AI technology with modern web development tools, the platform reduces the complexity of coding and improves productivity. The system also provides responsive designs and multiple color palettes to create visually appealing websites.

In the future, the system can be improved by adding more templates, advanced customization options, and JavaScript feature generation.